APPENDIX M

Machine intelligence

M.1. Space story

NASA's Deep Space 1 had an onboard "intelligent" control system that could do the planning. It was the first interplanetary mission to use an ion engine. It visited the asteroid 9969 Braille and the comet Borrelly. It was one of the first spacecraft to have Artificial-Intelligence software.

M.2. Introduction

Machine intelligence is a field in computer science where data is used to predict, or respond to, future data. It is closely related to the fields of pattern recognition, computational statistics, and Artificial Intelligence. The data may be historical or updated in real time. Machine learning is important in areas like facial recognition, spam filtering, and other areas where it is not feasible, or even possible, to write algorithms to perform a task. In many cases tedious work that had been done by humans can now be done by machines. Machine-learning algorithms can also generate new data, such as music or stories, by use of massive learning of sources.

Spacecraft are a natural fit for machine intelligence because all satellites need a high degree of autonomy. Even satellites that are under constant ground monitoring, such as geosynchronous communications satellites, benefit from high degrees of autonomy because problems can be very expensive to fix. In addition, operator time is expensive. For deep-space missions, communication time delays make autonomy even more important. Deep-space operations are also expensive, costing tens of millions of US dollars a year. Autonomous systems can reduce these costs. Most spacecraft today incorporate autonomy using boolean logic, the familiar if-else-then type of statements. The parameters for the logic are usually determined before launch. For example, boolean logic might check attitude errors measured by the sensors, and if a value is exceeded switch the spacecraft into a Sun-safe mode. Another example would be logic that checks the electrical current draw for a sensor or actuator, and switches devices if no current is detected.

This appendix will first give an overview of machine learning. The focus of the remaining parts of the appendix will be on deep learning. The first section then derives the Exclusive-OR (XOR) example showing how a neural network can replicate the XOR. Exclusive-OR is a fundamental operation used in computers with two inputs and one output. The output can take one of two states, depending on the four inputs.

An example of orbit determination using a neural network is given. Another example of using a neural network to get roll-and-pitch measurements from a static Earth sensor is described. An expert system will be demonstrated. Finally, an example of using reinforcement learning for an attitude maneuver with reaction wheels will be presented. The latter will be compared with an optimal maneuver using downhill Simplex.

M.3. Branches of machine intelligence

Machine learning as described above is the collecting of data, finding patterns, and doing useful things based on those patterns. We expand machine learning to include adaptive and learning control. These fields started independently but now are adapting technology and methods from machine learning. All control systems deal with uncertainty. For example, gain and phase margins are a measure of the tolerance a system has to uncertainty. Adaptive control systems go a step further and learn from their experiences. This is done via an algorithm. With machine learning, we usually forego creating an algorithm in advance and let the system find the method of relating inputs to outputs.

Fig. M.1 shows how we organize the technology of machine learning into a consistent taxonomy. You will note that the title encompasses three branches of learning; the whole subject area is called "Autonomous Learning". Which means, learning without human intervention during the learning process. This book is not solely about "traditional" machine learning. Other, more specialized books focus on any one of the machine-learning topics. For example, optimization is a tool used for trajectory planning and other spacecraft applications. The optimization software is often part of deep-learning training. It is used to find the optimal weights for the neurons for the neural networks.

There are three categories under Autonomous Learning. The first is *Control*. Feedback control is used to compensate for uncertainty in a system or to make a system behave differently from how it would normally behave. If there was no uncertainty you would not need feedback. For example, if you are kicking a ball at a teammate who is running, assume for a moment that you know exactly where the teammate will be at any time. This is the point of having predefined plays in sports. You know exactly where your teammate should be at a given time, so you can close your eyes, count, and just kick the ball to that spot. Assuming your teammate has good feet, you would have a 100% reception rate! More realistically, you watch the player, estimate their speed, and kick the ball. You are applying feedback to the problem. As stated, this is not a learning system. However, if now you practice the same play repeatedly, look at your success rate and modify the mechanics and timing of your kick using that information, you would have an adaptive control system, the box second from the top of the control list. Learning in control takes place in adaptive control systems and also in the general area of system identification.

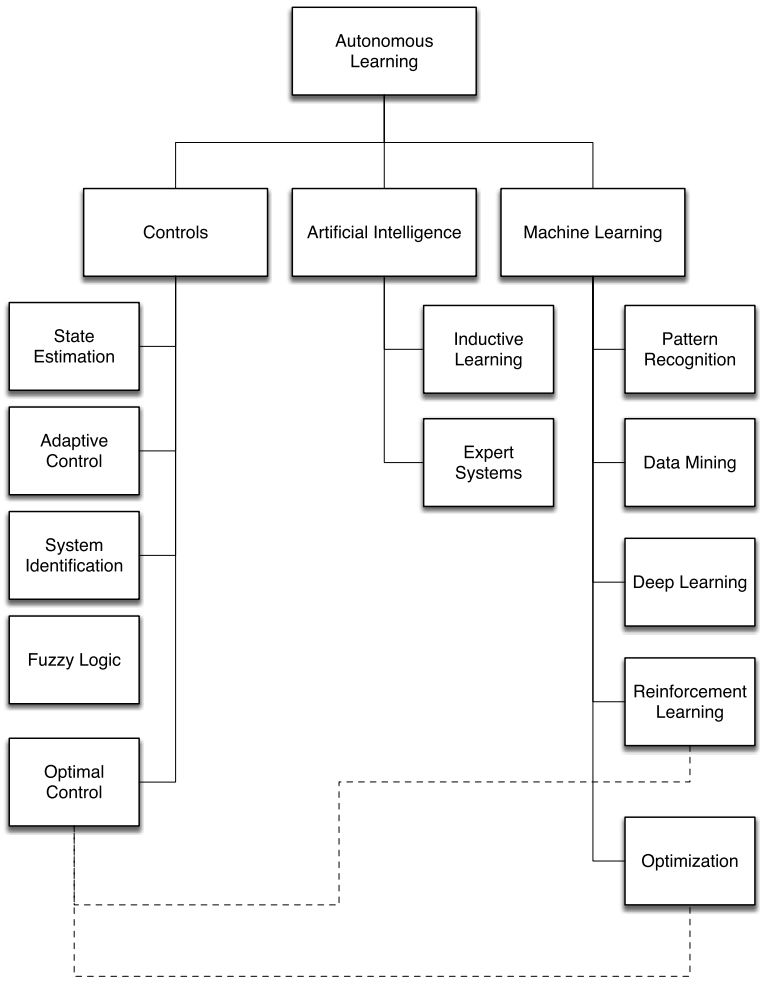


Figure M.1 Taxonomy of machine learning.

System identification is learning the parameters of the system. By system, we mean the data that represents anything and the relationships between elements of that data. For example, a particle moving in a straight line is a system defined by its mass, the force on that mass, its velocity, and its position. The position is the integral of the velocity over time and the velocity is the integral of the acceleration, which is the force divided by the mass. The concept of state has been discussed previously. Characteristics of a system can be compartmentalized into states and parameters. In general, the latter do not change in response to external influences.

Optimal control may not involve any learning. For example, full-state feedback produces an optimal control signal but does not involve learning. In full-state feedback, the combination of model and data tells us everything we need to know about the system. However, in more complex systems it is not possible to measure all the states or to know the parameters perfectly so some form of learning is needed to produce "optimal" or the best possible results.

The second category is what is usually considered true *Machine Learning*. This is the use of data to produce behavior that solves problems. Much of its background comes from statistics and optimization. The learning process may be done once in a batch process or continually in a recursive process. For example, in a stock-buying package, a person might have processed stock data for several years and used that to decide which stocks to buy. That software might not have worked well during a financial crash that had never been observed and for which there is no data. A recursive program would continuously incorporate new data. Pattern recognition and data mining fall into this category. Pattern recognition is used to find patterns in images. For example, the early AI Blocks World software could identify a block in its field of view. It could find one block of a particular color in a pile of blocks and determine its location. Data mining is taking large amounts of data and looking for patterns. For example, taking stock-market data and identifying companies that have strong growth potential. Classification techniques and fuzzy logic are also in this category.

The third category of autonomous learning is *Artificial Intelligence*. Machine learning traces some of its origins to Artificial Intelligence. Artificial Intelligence is the area of study whose goal is to make machines reason. While many would say the goal is "think like people" this is not necessarily the case. There may be ways of reasoning that are not similar to human reasoning but are just as valid. In the classic Turing test, Turing proposes that the computer only needs to imitate a human in its output to be a "thinking machine", regardless of how those outputs are generated. A system passes the Turing test if a human cannot tell if the responses are from a machine or another human being. In any case, intelligence generally involves learning so learning is inherent in many Artificial-Intelligence technologies such as inductive learning and expert systems. Our diagram includes the two techniques of inductive learning and expert systems.

M.4. Stored command lists

Spacecraft are often operated by commands sent from the ground station. A sequence of commands is usually required to perform a spacecraft operation. For example, to start a station-keeping maneuver with hydrazine thrusters the commands might be

1. Turn on the catalyst-bed heaters for the hydrazine thrusters.
2. Turn on the thrusters.
3. Turn on the rate gyros.

4. Switch to station-keeping attitude control mode.
5. Turn on the delta-V thrusters.

Each command is sent in sequence. Positive verification that the command was loaded in the spacecraft is usually required before sending the next command. This is known as handshaking and requires that telemetry be sent from the spacecraft and received at the ground station,

A degree of spacecraft autonomy can be achieved by storing these lists onboard the spacecraft. In this way a complex sequence can be performed with a single ground command. This reduces the workload on the ground station. In the case of the station-keeping sequence, the ground would initiate the sequence and then initiate the start of the burn. In this way the operators can make sure that all of the commands were executed properly. The next step would be to have the spacecraft check to see if a command was properly implemented. For example, it could check the current draw on the gyros before proceeding to the next step. A command-execution engine would eliminate the need for writing custom logic for each command list. The next step is to implement an expert system for performing command sequences. The expert system would be able to assemble a command list from a goal for the operation.

M.5. Deep Space 1

The Remote Agent experiment on the NASA Deep Space 1 mission provided a degree of autonomy on the spacecraft [1]. The spacecraft is shown in Fig. M.2. The mission also tested other technologies such as autonomous navigation and ion propulsion.

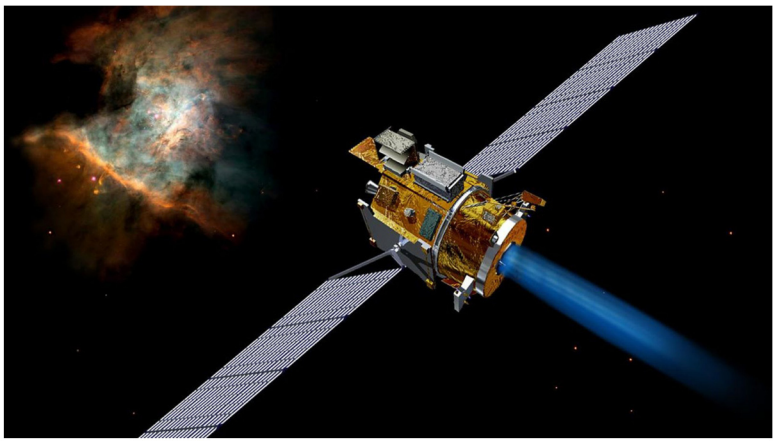


Figure M.2 The Deep Space 1 spacecraft. Image courtesy of NASA.

The architecture is shown in Fig. M.3. Real-time software is the flight software system that does conventional operations. The Remote Agent is managing that system,

much like ground controllers would do. The experiment manager operates the Remote Agent to do a series of experiments as part of the test program. The planner/scheduler maintains a database of goals for the mission.

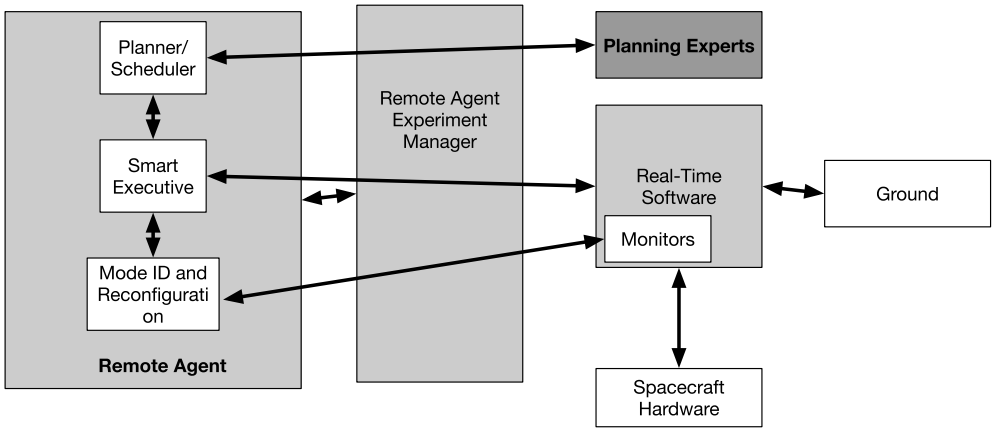


Figure M.3 Deep Space 1 autonomous flight system [2].

The Remote Agent enabled goal-based planning and robust failure recovery. The spacecraft was launched in October 1998 and the Remote Agent experiment was run in May of 1999. The levels of autonomy were

1. Single low-level command execution;
2. Time-stamped commands;
3. Single goal achievement with autorecovery;
4. Model-based state estimation and error detection;
5. Scripted plans with dynamic task decomposition;
6. Onboard back-to-back plan generation, execution, and recovery.

The first two are standard commanding. The others allowed the operators to set goals and have the system achieve those goals subject to operational constraints. For example, an operator could ask the spacecraft to take a picture and the system would make sure it could accomplish that goal. Default goals were also a part of the system, i.e., point an antenna at the Earth if the system had no other ongoing tasks. The planning was robust as it could automatically replan based on new information.

The Remote Agent Experiment was written in the Lisp programming language. Lisp is one of the first Artificial-Intelligence languages [3]. This presented challenges both because it was running on a processing-limited radiation-hard processor and had to interact with the C flight software. The experiment rigorously tested the software [4]. The goal was to produce a package that could be used on future missions.

M.6. Neural networks

Neural networks, or neural nets, are a popular way of implementing machine "intelligence". The idea is that they behave like the neurons in a brain. In this section, we will explore how neural nets work, starting with the most fundamental idea of a single neuron and working our way up to a multilayer neural net.

The simplest problem is a single neuron with two inputs. This is shown in Fig. M.4. This neuron has inputs x_1 and x_2, a bias b, weights w_1 and w_2, and a single output z. The activation function σ takes the weighted input and produces the output. In this diagram, we explicitly add icons for the multiplication and addition steps within the neuron, but in typical neural-net diagrams such as Fig. M.4, they are omitted.

$$z = \sigma(y) = \sigma(w_1 x_1 + w_2 x_2 + b) \tag{M.1}$$

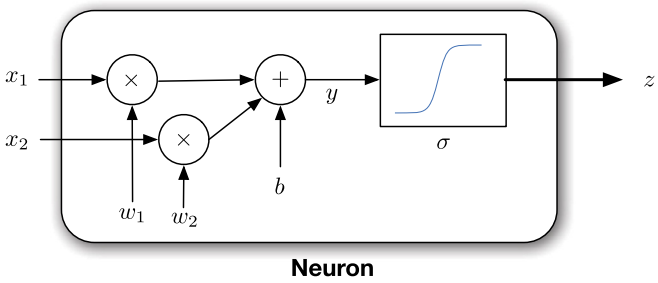


Figure M.4 A two-input neuron.

Let us compare this with a real neuron as shown in Fig. M.5. A real neuron has multiple inputs via the dendrites. Some of these branches can connect to the cell body through the same dendrite. The output is via the axon. Each neuron has one output. The axon connects to a dendrite through the synapse. Signals pass from the axon to the dendrite via a synapse.

There are numerous commonly used activation functions. Four are:

$$\sigma(y) = \tanh(y) \tag{M.2}$$

$$\sigma(y) = \frac{2}{1 - e^{-y}} - 1 \tag{M.3}$$

$$\sigma(y) = y \tag{M.4}$$

$$\sigma(y) = \begin{cases} y & y >= 0 \\ 0 & y < 0 \end{cases} \tag{M.5}$$

The exponential one is normalized and offset from zero so it ranges from -1 to 1. The last one, which simply passes through the value of y, is called the *linear* activation

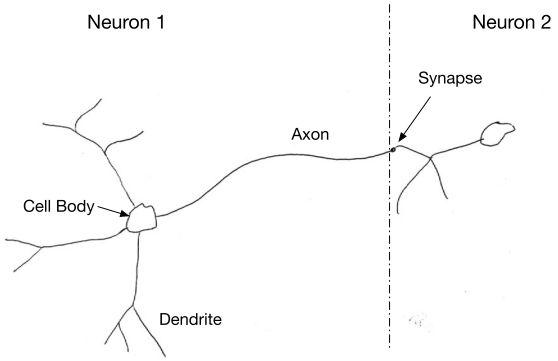


Figure M.5 A neuron connected to a second neuron. A real neuron can have 10 000 inputs!

function. The following code in the script OneNeuron.m computes and plots four activation functions for an input q. Fig. M.6 shows the four activation functions on one plot.

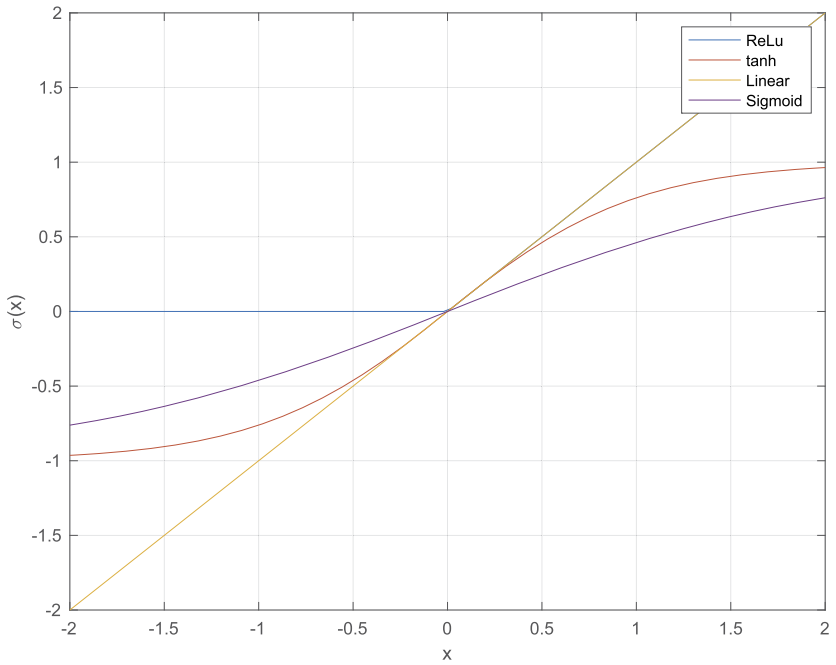


Figure M.6 Four activation functions.

Activation functions that saturate, that is reach a value of input after which the output is constant or changes very slowly, model a biological neuron that has a maximum firing

rate. These particular functions also have good numerical properties that are helpful in learning.

Let us look at a single-input neural net shown in Fig. M.7. This neuron is

$$z = \sigma(2x + 3) \tag{M.6}$$

where the weight w on the single input x is 2 and the bias b is 3. If the activation function is linear, the neuron is just a linear function of x,

$$z = y = 2x + 3 \tag{M.7}$$

Neural nets do make use of linear activation functions, often in the output layer. It is the nonlinear activation functions that give neural nets their unique capabilities.

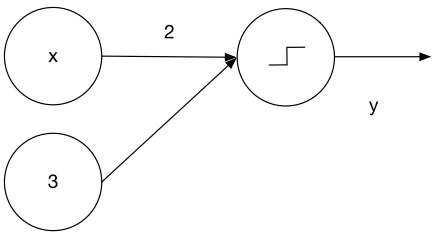


Figure M.7 A one-input neural net. The weight w is 2 and the bias b is 3. An artificial neuron consists of, at a minimum, the weight, and activation function. The bias is optional.

Let us look at the output with the above activation functions and also compare it with the threshold function The results are shown in Fig. M.8.

The tanh and exp are very similar. They put bounds on the output. Within the range $-3 \leq x < 1$ they return the function of the input. Outside those bounds they return the sign of the input, that is, they saturate. The threshold function returns zero if the value is less than 0 and 1 if it is greater than -1.5. The threshold is saying the output is only important, thus *activated*, if the input exceeds a given value. The other nonlinear activation functions say that we care about the value of the linear equation only within the given bounds. The nonlinear functions (but not steps) make it easier for the learning algorithms since the functions have derivatives. The binary step has a discontinuity at an input of zero so that its derivative is infinite at that point. Aside from the linear function (which is usually used on the output neurons), the neurons are just telling us that the sign of the linear equation is all we care about. The activation function is what makes this operation a neuron.

The XOR problem impeded the development of neural networks for some time before "deep learning", which is just a neural net with more than one layer of neurons, was developed. When the XOR problem was first studied it was shown that a single-layer neural network could not produce all four outputs from the two inputs. Look at

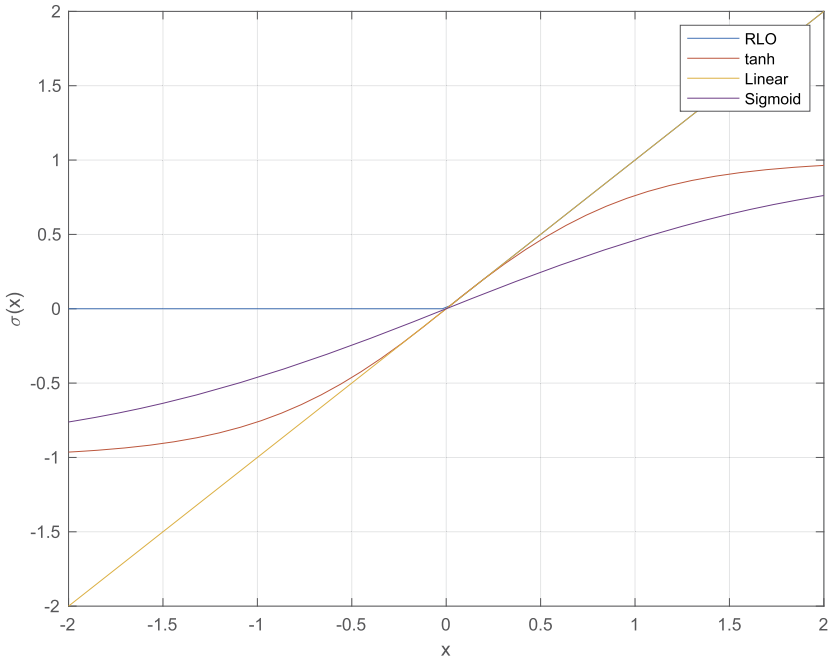


Figure M.8 The "linear" neuron compared to other activation functions from LinearNeuron.

Fig. M.9. The table on the left gives all possible inputs A and B and the desired outputs C. "Exclusive-Or" just means that if the inputs A and B are different, the output C is 1. The figure shows a single-layer network and a multilayer network, as in Fig. M.9, but with the weights labeled as they will be in the code. You can implement this easily, in just seven lines of code

```
a = 1;
b = 0;
if( a == b )
  c = 1
else
  c = 0
end
```

This type of logic was embodied in medium-scale integrated circuits in the early days of digital systems and vacuum tube-based computers before semiconductor-based computers. Try as you might, you cannot pick two weights and a bias on the single-layer network to reproduce the XOR. Marvin Minsky wrote proof that it was impossible.

A deep neural net, can reproduce the XOR. We will implement and train this network. We will explicitly write out the backpropagation algorithm that trains the

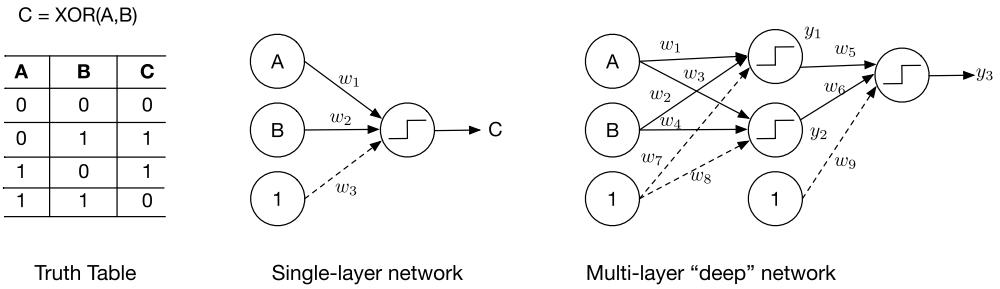


Figure M.9 Exclusive-Or (XOR) truth table and possible solution networks.

neural net from the four training sets given in Fig. M.9, i.e., (0,0), (1,0), (0,1), (1,1). We will use the tanh as the activation function in this example. tanh is a smooth function that can be differentiated, leading to an analytic result.

There are three neurons. The activation function for the hidden layer is the aforementioned hyperbolic tangent. The activation function for the output layer is linear.

$$y_1 = \tanh(w_1 a + w_2 b + w_7) \tag{M.8}$$

$$y_2 = \tanh(w_3 a + w_4 b + w_8) \tag{M.9}$$

$$y_3 = w_5 y_1 + w_6 y_2 + w_9 \tag{M.10}$$

Now, we will derive the backpropagation routine that is the learning method. The hyperbolic activation function is,

$$f(z) = \tanh(z) \tag{M.11}$$

Its derivative is

$$\frac{df(z)}{dz} = 1 - f^2(z) \tag{M.12}$$

In this derivation, we are going to use the chain rule. Assume that F is a function of y and y is a function of x. Then,

$$\frac{dF(y(x))}{dx} = \frac{dF}{dy}\frac{dy}{dx} \tag{M.13}$$

The error, at the output of the neural network, is the square of the difference between the desired output and the output. This is known as a quadratic error. It is easy to use because the derivative is simple and the error is always positive, making the lowest error the one closest to zero. Other error measures could be used.

$$E = \frac{1}{2}(c - y_3)^2 \tag{M.14}$$

The derivative of the error for w_j for the output node is

$$\frac{\partial E}{\partial w_j} = \left(y_3 - c\right) \frac{\partial y_3}{\partial w_j} \tag{M.15}$$

For the hidden nodes, it is

$$\frac{\partial E}{\partial w_j} = \psi_3 \frac{\partial n_3}{\partial w_j} \tag{M.16}$$

Expanding for all the weights,

$$\frac{\partial E}{\partial w_1} = \psi_3 \psi_1 a \tag{M.17}$$
$$\frac{\partial E}{\partial w_2} = \psi_3 \psi_1 b \tag{M.18}$$
$$\frac{\partial E}{\partial w_3} = \psi_3 \psi_2 a \tag{M.19}$$
$$\frac{\partial E}{\partial w_4} = \psi_3 \psi_2 b \tag{M.20}$$
$$\frac{\partial E}{\partial w_5} = \psi_3 y_1 \tag{M.21}$$
$$\frac{\partial E}{\partial w_6} = \psi_3 y_2 \tag{M.22}$$
$$\frac{\partial E}{\partial w_7} = \psi_3 \psi_1 \tag{M.23}$$
$$\frac{\partial E}{\partial w_8} = \psi_3 \psi_2 \tag{M.24}$$
$$\frac{\partial E}{\partial w_9} = \psi_3 \tag{M.25}$$

where

$$\psi_1 = 1 - f^2(n_1) \tag{M.26}$$
$$\psi_2 = 1 - f^2(n_2) \tag{M.27}$$
$$\psi_3 = y_3 - c \tag{M.28}$$
$$n_1 = w_1 a + w_2 b + w_7 \tag{M.29}$$
$$n_2 = w_3 a + w_4 b + w_8 \tag{M.30}$$
$$n_3 = w_5 y_1 + w_6 y_2 + w_9 \tag{M.31}$$

You can see from the derivation how this could be made recursive and applied to any number of outputs or layers. The eight adjustments at each step will be

$$\Delta w_j = -\eta \frac{\partial E}{\partial w_j} \tag{M.32}$$

where η is the update gain. It should be a small number. We only have four sets of inputs. We will apply them multiple times to get the XOR weights.

The backpropagation trainer needs to find the nine elements of w. The first step is to try the network with random weights and biases. As expected, the results of the neural network with random weights and biases are not good and it does not reproduce the XOR. After training, the neural network reproduces the XOR problem very well, as shown below. If you change the initial weights and biases, you may find that you get bad results. This is because the simple gradient method implemented here can fall into local minima from which it cannot escape. This is an important point about finding the best answer. There may be many good answers, which are local optimums, but there will be only one best answer. There is a vast body of research on how to guarantee that a solution is globally optimal. Other training systems, like simulated annealing and genetic algorithms, are less likely to become stuck in a local optimal.

Example M.1 shows the weights and biases converging and also shows the mean output error over all four inputs in the truth table going to zero. If you try other starting weights and biases this may not be the case. Other solution methods, such as Genetic Algorithms [5], Electromagnetism based [6], and Simulated Annealing [7] are less susceptible to falling into local minima but can be slow. A good overview of optimization specifically for machine learning is given by Bottou [8].

You might wonder how this compares to a set of linear equations. If we remove the activation functions we get

$$y_3 = w_9 + w_6 w_8 + w_5 w_7 + a(w_1 w_5 + w_3 w_6) + b(w_2 w_5 + 2_r w_6) \tag{M.33}$$

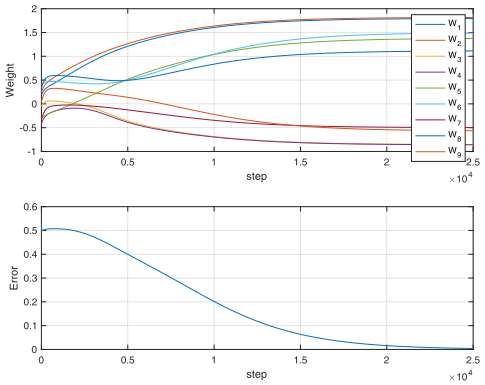
This reduces to just three independent coefficients

$$y_3 = k_1 + k_2 a + k_3 b \tag{M.34}$$

One is a constant, and the other two multiply the inputs. Writing the four possible cases in matrix notation we get

$$\begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix} = k_1 + \begin{bmatrix} 0 & 1 \\ 0 & 0 \\ 1 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} k_2 \\ k_3 \end{bmatrix} \tag{M.35}$$

```
% Training data - also the truth data
a        = [0 1 0 1];
b        = [0 0 1 1];
c        = [0 1 1 0];

% First try implementing random weights
w0       = [ 0.1892; 0.2482; -0.0495; -0.4162;
            -0.2710;...
             0.4133; -0.3476; 0.3258; 0.0383];
cR       = XOR(a,b,w0);

fprintf('\nRandom Weights\n')
fprintf('    a      b      c\n');
for k = 1:4
  fprintf('%5.0f %5.0f %5.2f\n',a(k),b(k),cR(k));
end

% Now execute the training
w        = XORTraining(a,b,c,w0,25000,0.001);
cT       = XOR(a,b,w);

fprintf('\nWeights and Biases\n')
fprintf('   Initial    Final\n')

for k = 1:9
  fprintf('%d  %7.4f %7.4f\n',k,w0(k),w(k));
end

fprintf('\nTrained\n')
fprintf('    a      b      c\n');
for k = 1:4
  fprintf('%5.0f %5.0f %5.2f\n',a(k),b(k),cT(k));
end
```




```
Random Weights
    a      b      c
    0      0   0.26
    1      0   0.19
    0      1   0.03
    1      1  -0.04

Weights and Biases
   Initial    Final
1   0.1892   1.7933
2   0.2482   1.8155
3  -0.0495  -0.8535
4  -0.4162  -0.8591
5  -0.2710   1.3744
6   0.4133   1.4893
7  -0.3476  -0.4974
8   0.3258   1.1124
9   0.0383  -0.5634

Trained
    a      b      c
    0      0   0.00
    1      0   1.00
    0      1   1.00
    1      1   0.01
```

Example M.1: XOR neural-network weights converge during training.

We can get close to a working XOR if we choose

$$\begin{bmatrix} k_1 \\ k_2 \\ k_3 \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \\ -1 \end{bmatrix} \tag{M.36}$$

This makes three out of four equations correct. There is no way to make all four correct with just three coefficients, as Minsky proved. The activation functions separate the coefficients and allow us to reproduce the XOR. This is not surprising because the XOR is not a linear problem.

M.7. Static Earth sensors

Earth sensors were some of the earliest spacecraft sensors. They are still very popular for CubeSats. Many early satellites were for communications or Earth observations so during most of the satellite life it stared at the Earth, which means zeroing the angles between the axis pointing at the Earth and the nadir vector. In many cases, the yaw error, the error around the nadir vector or vector to the Earth, was not important or could be controlled through the momentum of the satellite if it was a momentum-bias or dual-spin spacecraft. Earth sensors were either static, without moving parts, or scanning. Scanning was done by rotating the satellite, rotating a mirror using a motor, or vibrating a mirror with a torsion bar. Most sensors operated in the infrared wavelengths because the radiation at those wavelengths is more uniform than that in the visible bands and is unaffected by eclipses.

Fig. M.10 shows the attitude (also known as orientation) geometry for small angles from ideal Earth pointing. Roll is about the x-axis and pitch is about the y-axis.

Fig. M.11 shows how a static Earth sensor operates. The image on the left shows the Earth-sensor elements, represented by the circles centered on the Earth. The roll and pitch angles are zero. Each element sees part of the Earth and part of space so they all produce the same signal. The picture on the right shows the sensor with a roll and pitch angle. Some elements see just space, some see just the Earth, and some both the Earth and space but their output will be different from the zero angle cases. By comparing the outputs of the sensors we get a measurement of roll and pitch. The sensors see light in the CO_2 band, around 14 micrometers wavelength, so the Earth looks uniform.

Static Earth sensors from the 1960s had built-in logic to go from sensor outputs to roll and pitch. Given the hardware limitations, the algorithms were quite simple and could be implemented with a few transistors or medium-scale integrated circuits. In this section, we show how a neural network can be trained to take the sensor inputs and return roll-and-pitch measurements.

Example M.2 shows the neural-network training and execution.

The first part of the script generates the training data. It calls ISSOrbit to get the Keplerian orbital elements for the International Space Station (ISS). It then gets the default data structure from StaticEarthSensor. The script then converts orbital elements into position and velocity vectors using RVOrbGen and computes the quaternion from the Earth-centered inertial frame (ECI) to the local vertical/local lorizontal (LVLH), which is the normal Earth-pointing orientation. We then create a four-element Earth-sensor model, replacing the default fields in d. This line of code draws the sensor.

The top plot shows the performance over the range of input roll-and-pitch angles. The middle plot shows the training data. The last plot shows roll-and-pitch measurement over the test orbit when the true roll is two degrees and the true pitch is zero.

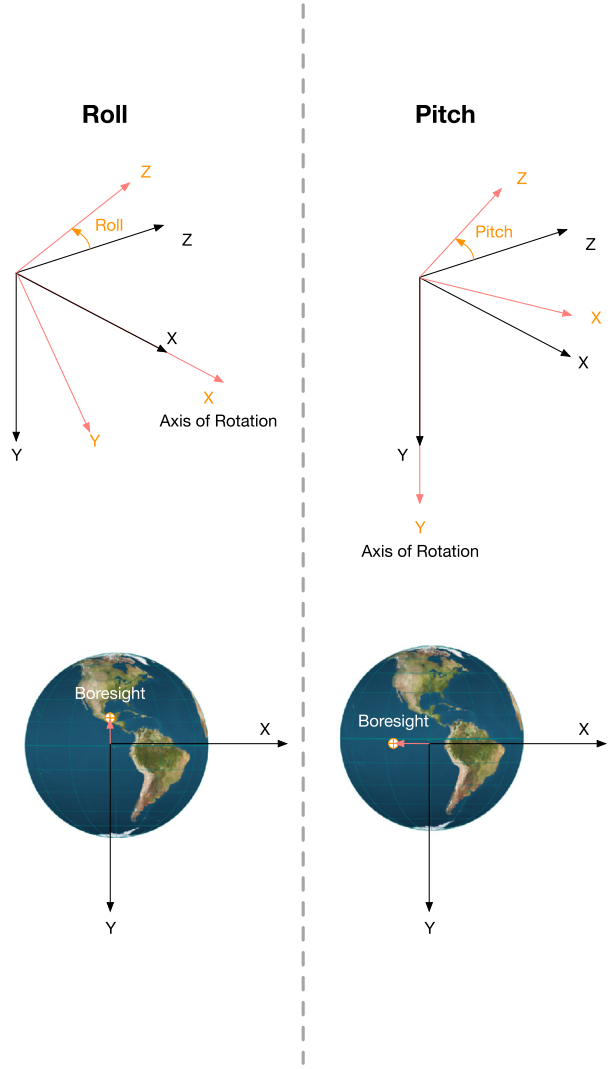
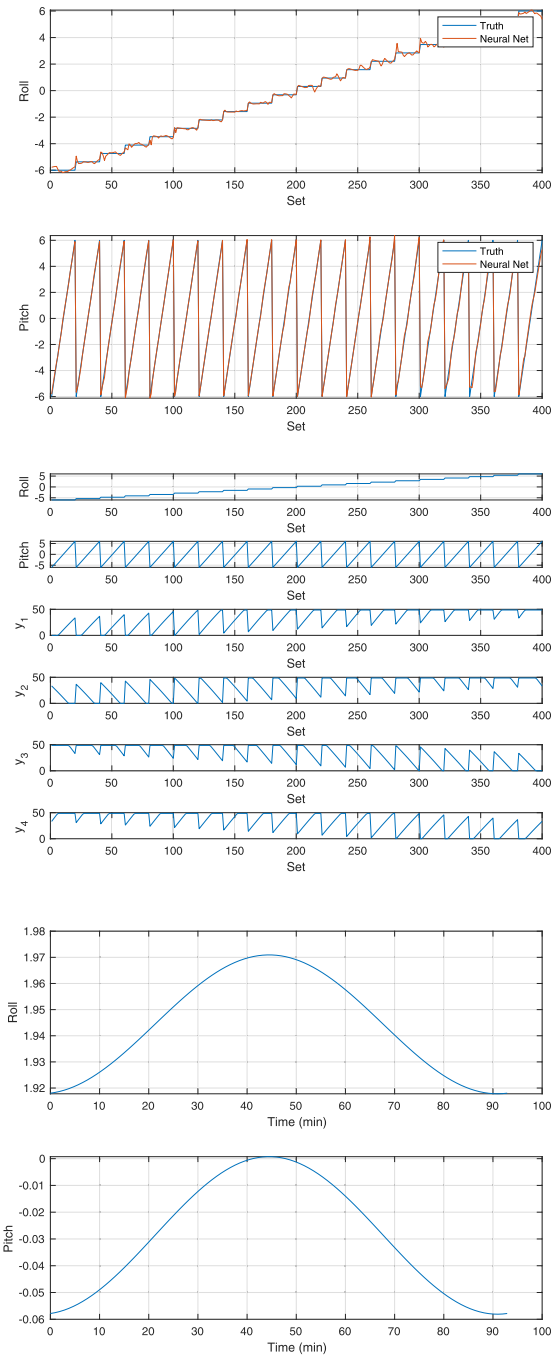


Figure M.10 Attitude geometry showing roll and pitch.

Since this neural network was only trained at one point in the orbit, that is at one altitude, it will not match roll and pitch exactly over the entire orbit.

Fig. M.12 shows the Earth sensor and Fig. M.13 the training GUI from MATLAB®. The GUI shows the structure of the neural network. Fig. M.12 shows the Earth sensor and the training GUI from MATLAB. The GUI shows the structure of the neural network.

```
degToRad     = pi/180;
rE           = 6378.165;
[el,jD0]     = ISSOrbit;
d            = StaticEarthSensor;
[r,v,t]      = RVOrbGen(el);
rMean        = mean(Mag(r));
qECIToLVLH   = QLVLH(r,v);
d.el         = 64*ones(1,4)*degToRad;
d.az         = [0 pi/2 pi 3*pi/2] + pi/4;
d.pAng       = 4*[  1    1   -1   -1
                    1   -1   -1    1];

n            = 20;
roll         = linspace(-6,6,n);
pitch        = linspace(-6,6,n);
i            = 1;
y            = zeros(4,n*n);
x            = zeros(2,n*n);

StaticEarthSensor(qECIToLVLH(:,1),r(:,1),d)

for j = 1:n
  for k = 1:n
    rJ           = roll(j);
    pK           = pitch(k);
    mRoll        = [1 0 0;0 CosD(rJ) -SinD(rJ);0
          SinD(rJ) CosD(rJ)];
    mPitch       = [CosD(pK) 0 -SinD(pK);0 1 0;
          SinD(pK) 0 CosD(pK)];
    qLVLHToBody = Mat2Q(mRoll*mPitch);
    qECIToBody   = QMult(qECIToLVLH(:,1),
          qLVLHToBody);

    y(:,i)       = StaticEarthSensor(qECIToBody,r
          (:,1),d);
    x(:,i)       = [roll(j);pitch(k)];
    i            = i + 1;
  end
end

% Neural net training
net          = feedforwardnet(20); % Generate the
      neural net structure

net          = configure( net, y, x ); % Configure
      based on inputs and outputs
net.layers{1}.transferFcn = 'poslin'; % Set as
      purelin
net.name     = 'Earth Sensor';
net          = train(net,y,x); % Train the network
c            = sim(net,y); % Simulate the neural
      network
leg          = {'Truth' 'Neural Net'};

PlotSet(1:size(c,2),[x;c],'x label','Set','y
      label',{'Roll' 'Pitch'},...
       'figure title','Neural Network','plot set
            ',{[1 3],[2 4]},...
       'legend',{leg leg});

yL = {'Roll' 'Pitch' 'y_1' 'y_2' 'y_3' 'y_4'};
PlotSet(1:size(c,2),[x;y],'x label','Set','y
      label',yL,'Neural Network Data')

%% Testing
n     = length(t);
roll  = 2;
pitch = 0;
c     = zeros(2,n);
for k = 1:n
  rJ           = roll;
  pK           = pitch;
  mRoll        = [1 0 0;0 CosD(rJ) -SinD(rJ);0
        SinD(rJ) CosD(rJ)];
  mPitch       = [CosD(pK) 0 -SinD(pK);0 1 0;SinD(
        pK) 0 CosD(pK)];
  qLVLHToBody = Mat2Q(mRoll*mPitch);
  qECIToBody   = QMult(qECIToLVLH(:,k),qLVLHToBody
        );
  y            = StaticEarthSensor(qECIToBody,r(:,
        k),d);
  c(:,k)       = sim(net,y');
end

[t,tL]   = TimeLabl(t);
s        = sprintf('Roll = %8.2f deg Pitch = %8.2f
      deg',roll,pitch);

PlotSet(t,c,'x label',tL,'y label',{'Roll' '
      Pitch'},'figure title',s)
```



Example M.2: Neural network trained to produce pitch-and-roll measurements from sensor measurements.

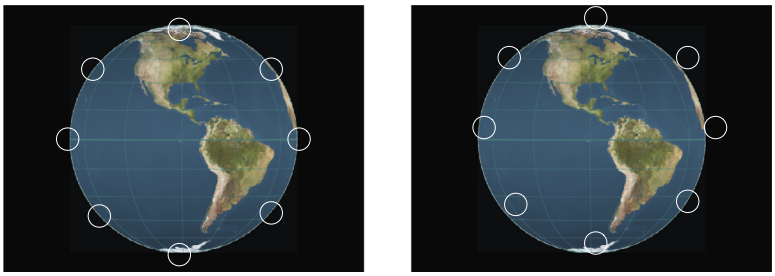


Figure M.11 Static Earth-sensor operation. The circles represent the sensor elements. The left shows the geometry with zero roll and pitch. The right shows the geometry with roll-and-pitch angles.

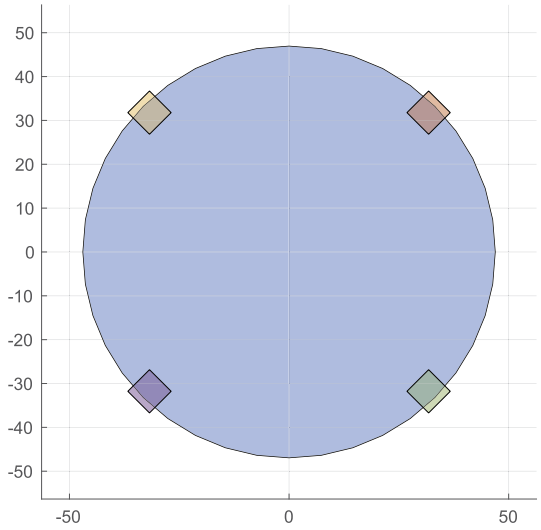


Figure M.12 The Earth sensor.

M.8. Expert systems

Expert systems are a way of embodying knowledge about a system in a database and a means for making decisions based on that data. Traditionally, this has been done by using logical statements in computer code. For example, a spacecraft might check an electrical-current measurement for a reaction wheel to see if it is drawing power.

```
{
        if( measuredCurrent > 0 )
         % Use the reaction wheel
        else
         % Use a backup wheel
```

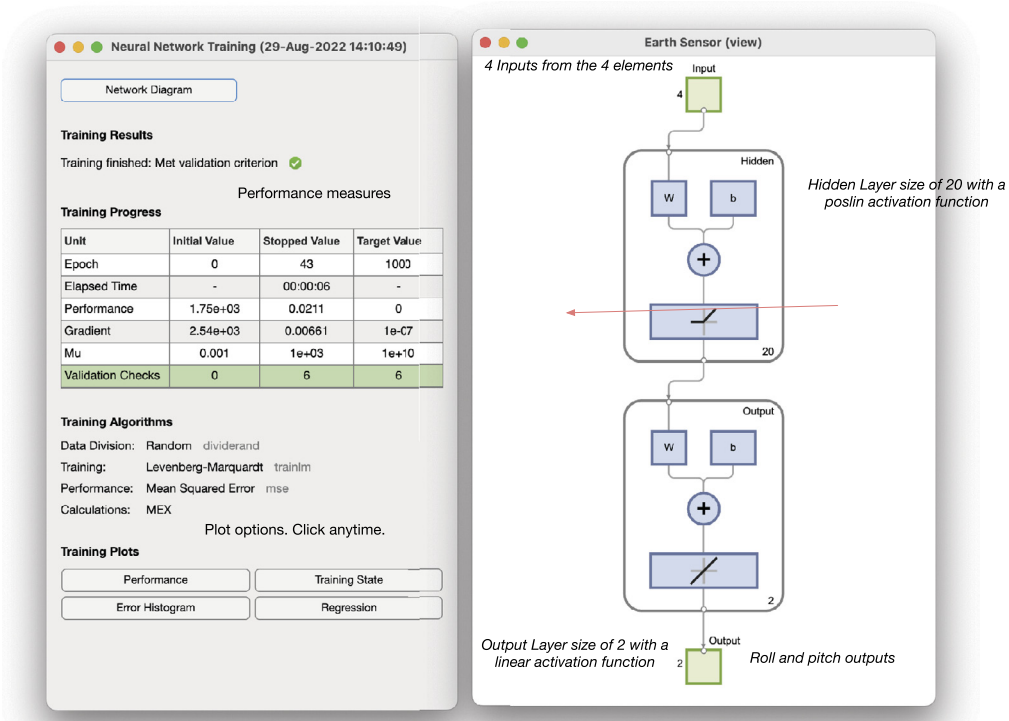


Figure M.13 The training GUI.

```
        end
}
```

This can work well for small numbers of these constructs. For very large numbers of logical statements, the software becomes very difficult to maintain and test. Expert systems are an elegant way to provide such capabilities.

There are two broad classes of expert systems, rule-based and case-based. Rule-based systems have a set of rules that are applied to make a decision. These systems provide a way of automating the process when decision-making involves hundreds or thousands of rules. Case-based systems decide by example, that is, a set of predefined cases. NASA CLIPS (C Language Integrated Production System) is a rule-based language that was developed by NASA's Johnson Space Center. NASA Deep Space 1 [9] used Remote Agents (RA) that employed a rule-based expert system for mission planning and execution and fault detection.

Learning in the context of an expert system depends strongly on the configuration of the expert system. There are three primary methods, which vary in the level of

autonomy of learning and the average generalization of the new knowledge of the system.

The least autonomous method of learning is the introduction of new rule sets in simple rule-based expert systems. Learning of this sort can be highly tailored and focused, but is done entirely at the behest of external teachers. In general, very specific rule-based systems with extremely general rules tend to have issues with edge cases that require exceptions to their rules. Thus, this type of learning, while easy to manage and implement, is neither autonomous nor generalizable.

The second method is fact gathering. The expert system makes decisions based on the known cause-and-effect relationships along with an evolving model of the world; learning then is broken up into two subpieces. Learning new cause-and-effect system rules is very similar to the type of learning described above, requiring external instruction, but can be more generalizable (as it is combined with more general world knowledge than a simple rule-based system might have). Learning new facts, however, can be autonomous and involves the refinement of the expert system's model of reality by increasing the amount of information that can be taken advantage of by automated reasoning systems.

The third method is fully autonomous-based reasoning, where actions and their consequences are observed, leading to inferences about what prior and action combinations lead to what results. For instance, if two similar actions result in positive results, then those priors that are the same in both cases can begin to be inferred as necessary preconditions for a positive result from that action. As additional actions are seen, these inferences can be refined and confidence can increase in the predictions made.

The three methods are listed in increasing difficulty of implementation. Adding rules to a rule-based expert system is quite straightforward, though rule dependencies and priorities can become complicated. Fact-based knowledge expansion in automated reasoning systems is also fairly straightforward, once suitably generic sensing systems for handling incoming data are set up. The third method is by far the most difficult; however, rule-based systems can incorporate this type of learning. In addition, more general pattern-recognition algorithms can be applied to training data (including online, unsupervised training data) to perform this function, learning to recognize, e.g., with a neural network, patterns of conditions that would lead to positive or negative results from a given candidate action. The system can then check possible actions against these learned classification systems to gauge the potential outcome of the candidate's actions.

The following example explores case-based reasoning systems. This is a collection of cases with their states and values given by strings. The code in this chapter can be made to learn by feeding back the results of new cases into the case-based system.

The knowledge base consists of states, values, and production rules. There are four parts to a new case: the case name, the states, values, and the outcome. A state can have multiple values.

The state catalog is a list of all of the information that will be available to the reasoning system. It is formatted as states and state values.

The default catalog is shown below for a reaction-wheel control system. A MATLAB cell array of acceptable or possible values for each state follows the state definition

```
{
        {'wheel-turning'},      {'yes','no'};
        {'power'},              {'on','off'};
        {'torque-command'},     {'yes','no'}
}
```

The database of cases is designed to detect failures. There are three things to check to see if the wheel is working. If the wheel is turning and power is on and there is a torque command then it is working. The wheel can be rotating without a torque command or with the power off because it would just be spinning down from prior commands. If the wheel is not turning the possibilities are that there is no torque command or the power is off.

Once cases are defined from the catalog, the system can be tested The function CBREngine implements the case-based reasoning engine. The algorithm finds the total fraction of the cases that match to determine if the example matches the stored cases. The engine is matching values for states in the new case against values for states in the case database. It weights the results by the number of states. If the new case has more states than an existing case, the result is the number of states in the database case divided by the number of states in the new case. If more than one case matches the new case, and the outcomes for the matching cases are different, the outcome is declared "ambiguous". If they are the same, it gives the new case that outcome. The case names make it easier to understand the results.

The demo script, Example M.3 builds the system and runs some cases. The script matches two cases but because their outcome is the same, the wheel is declared working. For the wheel power, the power could be on or off, hence it is declared ambiguous.

This example is for a very small case-based expert system with a binary outcome. Multiple outcomes can be handled without any changes to the code. However, the matching process is slow as it cycles through all the cases.

M.9. Reinforcement learning

M.9.1 Introduction

Reinforcement learning trains an agent to perform a task within a specified environment. The architecture of an agent is shown in Fig. M.14. The policy is the mechanism that turns observations, sensor measurements for example, into actions, such as control torques. A control system is an agent that is trained by the control-system designer,

```
system = BuildExpertSystem( [], 'id',1,...
'catalog state name','wheel-turning',...
'catalog value',{'yes','no'},...
'id',2,...
'catalog state name','power',...
'catalog value',{'on' 'off'},...
'id',3,...
'catalog state name','torque-command',...
'catalog value',{'yes','no'},...
'id',1,...
'case name', 'Wheel operating',...
'case states',{'wheel-turning', 'power', 'torque-
    command'},...
'case values',{'qyes' 'on' 'yes'},...
'case outcome','working',...
'id',2,...
'case name', 'Wheel power ambiguous',...
'case states',{'wheel-turning', 'power', 'torque-
    command'},...
'case values',{'yes' {'on' 'off'} 'no'},...
'case outcome','working',...
'id',3,...
'case name', 'Wheel broken',...
'case states',{'wheel-turning', 'power', 'torque-
    command'},...
'case values',{'no' 'on' 'yes'},...
'case outcome','broken',...
'id',4,...
'case name', 'Wheel turning',...
'case states',{'wheel-turning', 'power'},...
'case values',{'yes' 'on'},...
'case outcome','working',...
'match percent',80);

newCase.state  = {'wheel-turning', 'power', '
    torque-command'};
newCase.values = {'yes','on','no'};
newCase.outcome = '';

[newCase.outcome, pMatch] = CBREngine( newCase,
    system );

fprintf(1,'New case outcome: %s\n\n',newCase.
    outcome);

fprintf(1,'Case ID Name                                                    Percentage Match\n');
for k = 1:length(pMatch)
  fprintf(1,'Case %d: %-30s %4.0f\n',k,system.
      case(k).name,pMatch(k)*100);
end
```

```
New case outcome: working

Case ID Name                                                    Percentage Match
Case  1: Wheel working                                          67
Case  2: Wheel power ambiguous                                  67
Case  3: Wheel broken                                           33
Case  4: Wheel turning                                          44
```

Example M.3: Case-based expert system for reaction-wheel fault detection.

usually through the computation of a set of parameters with a fixed structure, such as a PID controller. A PID that learned its gains through repeated tests would fit into the architecture shown in Fig. M.14. A reinforcement-learning system can go beyond this by creating the algorithm itself. The policy is often a multilayer, or deep, neural network. Through training, the neuron weights and biases are chosen to produce a policy that maximizes the reward. The reward is based on how well the agent achieves the objective. For example, an agent would get a big reward if it maintains zero pointing errors.

Agents can be categorized into three groups:

- Discrete actions and discrete observations;

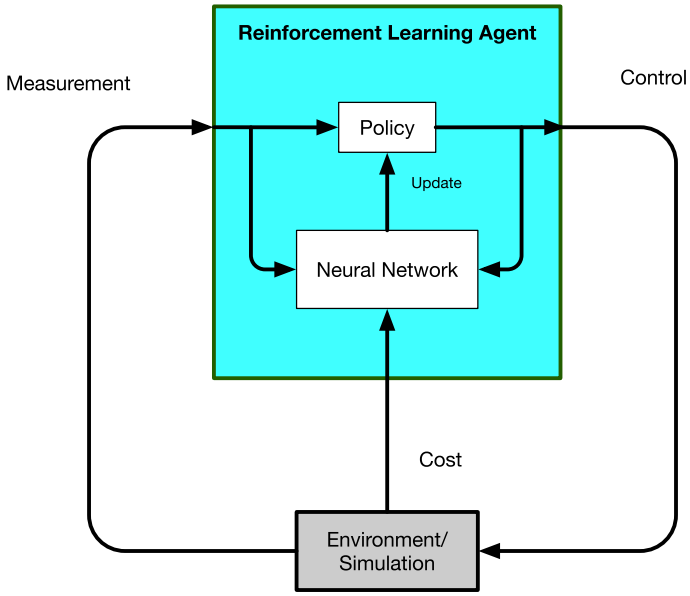


Figure M.14 Reinforcement-learning architecture.

- Discrete actions and continuous observations;
- Both continuous actions and observations.

A bang-zero-bang controller, such as the one implemented in the phase-plane logic on the Space Shuttle Orbiter, is an example of discrete actions and continuous observations. Continuous actions and observations would describe any conventional control system.

The maneuver algorithm will use a Deep Deterministic Policy Gradient (DDPG) algorithm [10]. It is a model-free, online, offpolicy reinforcement-learning method. It works with continuous observations and actions. The policy maps observations to actions. The policy is trained using data from simulations. The policy is updated with the latest observations. Offpolicy agents create a buffer of observations and use that to update the policy. It can always use old observation/action pairs. A DDPG agent is an actor–critic reinforcement-learning agent that searches for an optimal policy.

Within a DDPG agent, there are actors and critics. An actor performs the action, in this case, it commands the reaction-wheel torque, based on the observations. A critic evaluates the performance. Both the actor and critic are neural networks.

Reinforcement learning is based on maximizing a reward, which is no different from minimizing a cost in an optimal control problem. For a spacecraft maneuver the reward is based on the following:

1. Reaching the desired state;
2. The time it takes to reach the final state;

3. The amount of momentum the wheels need to store.

The desired state includes both the desired quaternion and the desired angular rate. During the maneuver, only the third criterion can be evaluated. The second criteria are evaluated once the first criteria are achieved.

M.9.2 Optimal attitude trajectory

This approach is similar to constrained optimization using a direct optimization method. The constraint is the final state, which must be the target. It is usually easiest to specify a fixed terminal time although time can be a value that the algorithm finds. The trajectory is broken up into segments with constant control on each segment. The optimizer then adjusts the controls until the final state is met. If the optimizer has difficulty, it can be given more time to find the solution. The cost is the sum of the magnitudes of the applied torque. The control is also constrained. The optimization needs a reasonable guess of the control to start. In this case, a bang-bang maneuver for a single axis is a reasonable first guess. The torque is a positive constant and then a negative constant. The control is bounded by the maximum-allowable torque.

Example M.4 shows the optimization code. The terminal state is the constraint. The software uses a fixed end time. The body rates must be zero and the quaternion must match the target quaternion at the terminal condition. The constraint is a six-vector that is the desired terminal attitude and angular rates. The attitude part is computed from the last three elements of the delta quaternion computed by

$$q_\delta = q^T \otimes q_T \tag{M.37}$$

where q is the terminal quaternion at the selected end time. If the delta quaternion is

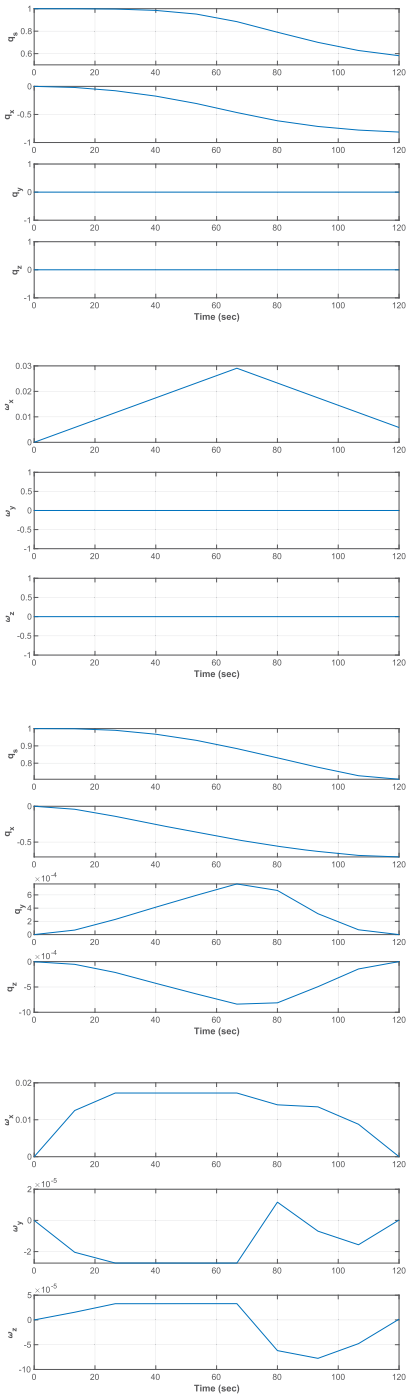
$$q_\delta = \begin{bmatrix} s \\ v \end{bmatrix} \tag{M.38}$$

the constraint equation is

$$c = \begin{bmatrix} v \\ \omega \end{bmatrix} \tag{M.39}$$

Ideally, this is a zero vector. Using the entire quaternion leads to singularities when the Hessian matrix is computed. fmincon is used with the interior-point method. This method computes a numerical Hessian matrix. The Hessian provides information on the curvature of the problem thus speeding convergence. Interior-point methods traverse the feasible region's interior, not the boundary. Only ten segments are used. That is, the control only changes ten times throughout the optimization. Larger numbers of points slow the process and may not get better results. In this case, the problem has a known solution with just two segments.

```
% Number of control segments
n               = 10;
torqueMax       = 0.01;
d.dRHS          = RHSGyrostat; % The data structure
d.n             = n; % Number of decision variable
      increments
d.tEnd          = 2*60;
d.t             = linspace(0,d.tEnd,n);
theta           = pi/2;
qEnd            = AU2Q(theta,[1;0;0]);
d.xEnd          = [qEnd;zeros(6,1)];

% Initial state
x               = [1;zeros(9,1)];
d.x             = x;

% fmincon options
opts = optimset(  'Display','iter-detailed',...
 'TolFun',1e-4,...
 'algorithm','interior-point',...
 'TolCon',1e-4,...
 'MaxFunEvals',15000);

% The cost is the sum of the torque magnitudes
costFun  = @(x) MnvrCost(x,d);

% The numerical integration of the state is in
      the constraint function
constFun = @(x) MnvrConst(x,d);

% x-axis maneuver
torque = theta*d.dRHS.inr(1,1)/(d.tEnd/2)^2;
oN = ones(1,n/2);
u0 = [torque*[-oN,oN];zeros(2,n)];

% Lower and upper bounds for the control
lB = -torqueMax*ones(3,n);
uB =  torqueMax*ones(3,n);

% Find the optimal decision variable
u = fmincon(costFun,u0,[],[],[],[],lB,uB,constFun
      ,opts);

costBase    = MnvrCost(u0,d)
costOptimal = MnvrCost(u,d)

%% Run the simulations
SimulationRWA( x, d.t, d.dRHS, u0 );
SimulationRWA( x, d.t, d.dRHS, u );


%% Maneuver cost
function cost = MnvrCost( u, d )

magU = Mag(u);
cost = sum(magU);

end

%% Maneuver nonlinear constraint
function [cIn, cEq] = MnvrConst( u, d )

xP  = SimulationRWA( d.x, d.t, d.dRHS, u );

% We don't have any nonlinear inequality
      contraints
cIn = [];

cQ  = QMult(QPose(xP(1:4,end)),d.xEnd(1:4));

cEq = [abs(cQ(2:4));xP(5:7,end)];

end
```



Example M.4: Optimal attitude trajectory.

The optimizer iterations are shown below. The first column is the number of iterations. The second, f(x) is the cost, in this case, the sum of the torque magnitudes. The second column is the total function count. The third column, Feasibility, is how well it is matching the landing constraint of zero quaternion error and angular rates. Ideally, it is zero when the constraint is matched. The First-Order Optimality is how close the solution is to the optimal solution. The Norm step size is the Norm (essentially magnitude) of the control step it is taking. It uses only about 1/3 of the maximum allowable steps.

```
                                                First-order      Norm of
 Iter F-count            f(x)  Feasibility      optimality          step
    0       31    4.363323e-03    1.641e-01    1.044e-02
    1       62    3.979051e-03    1.514e-03    1.069e+01    1.693e-03
    2       93    3.043139e-03    2.629e-06    3.144e+00    6.527e-04
    3      165    3.043136e-03    1.147e-06    1.073e-02    4.323e-09
    4      196    2.796689e-03    2.095e-01    3.722e+00    7.572e-04
    5      229    2.756049e-03    1.569e-01    3.274e+00    2.598e-04
    6      263    2.488857e-03    1.372e-01    2.905e+00    4.419e-04
    7      300    2.715273e-03    8.616e-04    1.089e-02    2.207e-04
    8      340    2.706251e-03    1.366e-06    5.009e-01    1.217e-04
    9      379    2.678182e-03    8.713e-03    5.014e-01    3.089e-05
   10      411    2.657084e-03    8.690e-03    1.498e+00    3.041e-05

  140     4990    2.586397e-03    4.436e-06    7.234e-01    4.630e-09
  141     5022    2.586396e-03    4.677e-06    7.753e-01    4.633e-09
  142     5057    2.586385e-03    5.219e-06    7.342e-01    9.255e-09
  143     5089    2.586384e-03    5.024e-06    7.342e-01    1.853e-08
  144     5126    2.586372e-03    8.224e-06    7.898e-01    9.244e-09
  145     5158    2.586371e-03    8.472e-06    8.136e-01    9.261e-09
  146     5193    2.586398e-03    3.027e-06    4.869e-01    1.850e-08
  147     5234    2.586396e-03    3.020e-06    4.875e-01    2.027e-09

The optimization stopped because the relative changes in all elements of the state $x$
     are
less than options.StepTolerance = 1.000000e-10, and the relative maximum constraint
violation, 3.020445e-06, is less than options.ConstraintTolerance = 1.000000e-04.

costBase =

    0.0044

costOptimal =

    0.0026
```

The list only shows the first eleven and last eight steps. The cost of the optimal maneuver is almost half that of the simple maneuver. As the plots show, the spacecraft does not do a single-axis maneuver. The rates for the other axes are very small but nonzero.

M.9.3 Single-axis optimal attitude trajectory

A single-axis problem is useful to explore reinforcement learning. The problem is to change the angle by a predetermined amount. The dynamical equations are

$$\dot{\theta} = \omega \tag{M.40}$$

$$\dot{\omega} = u \tag{M.41}$$

The DDPG agent applies reinforcement learning to this problem. The reinforcement learning system must maximize the reward that is a combination of

- The maximum torque;
- The rate error at the end;
- The angle error.

The reward algorithm is as follows.

```
% Check terminal condition
angError     = this.State(1)-this.thetaTarget;
IsDone       = abs(angError) < this.thetaError
    ;
this.IsDone = IsDone;

% When done we want the angular rate to be
    zero
if( IsDone )
  c = 100*exp(-1500*Mag(this.State(2)));
else
  c = 0;
end

% Negative reward for torque and error from
    the target
Reward = -0.001*torque^2 - 0.01*angError^2 +
    c;
```

When the rate error is zero, c is 100. At each step, the reward for being far from the target and for large torques is negative. The weights on the various components will determine the overall performance

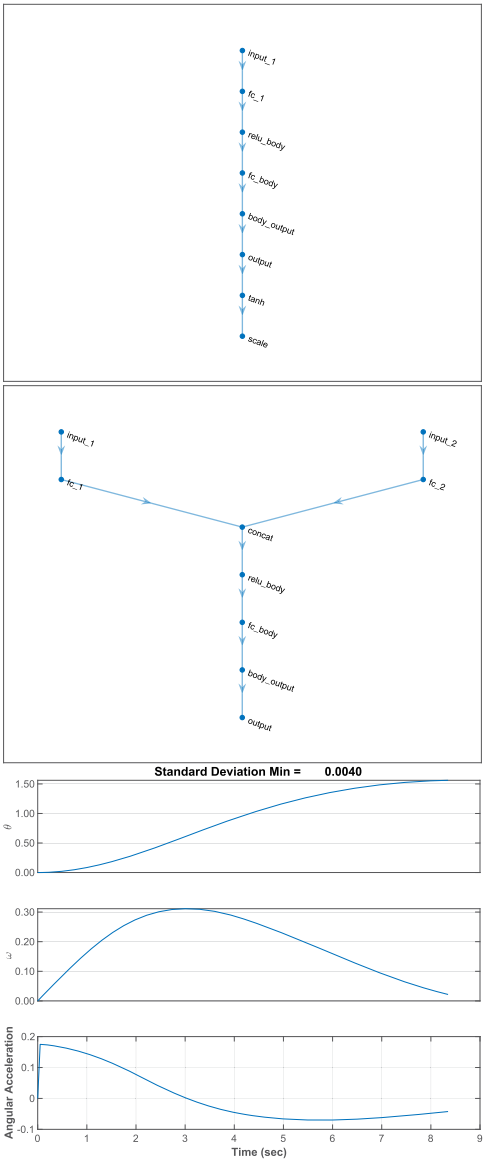
DDPG is an actor–critic approach. The actor selects an action based on a policy while the policy is evaluated by a critic using a value function [10].

The training window is shown in Fig. M.15. The dark blue line, which is the average reward since the start, should converge on the best reward.

Though it is not easy to see, close inspection shows that Q0 does converge to the mean reward.

The first diagram is a flow diagram for the actor network, the one that commands the angular acceleration. The second is the critic network that evaluates the performance. The minimum noise, and other parameters, result in a linear control law.

```
env      = OneAxisClass;

obsInfo = getObservationInfo(env);
actInfo = getActionInfo(env);
disp(obsInfo);
disp(actInfo);

initOpts   = rlAgentInitializationOptions('
        NumHiddenUnit',128);
agent      = rlDDPGAgent(obsInfo,actInfo,initOpts)
        ;

% Customize the agent
agent.AgentOptions.NoiseOptions.
        StandardDeviationDecayRate = 1e-5;
agent.AgentOptions.NoiseOptions.
        StandardDeviationMin = 0.02*agent.
        ActionInfo.UpperLimit;
agent.AgentOptions.CriticOptimizerOptions =
        rlOptimizerOptions('LearnRate',1e-4);
agent.AgentOptions.ActorOptimizerOptions =
        rlOptimizerOptions('LearnRate',1e-4);
actorNet   = getModel(getActor(agent));
criticNet = getModel(getCritic(agent));

NewFig('Actor Network')
plot(layerGraph(actorNet))
NewFig('Critic Network')
plot(layerGraph(criticNet))
disp('Critic Network')
disp(criticNet.Layers)
disp('Actor Network')
disp(actorNet.Layers)

doTraining = true;

maxsteps = 200;
maxepisodes = 1500;
trainingOpts = rlTrainingOptions(...
    'MaxEpisodes',maxepisodes,...
    'MaxStepsPerEpisode',maxsteps,...
    'Verbose',true,...
    'Plots','training-progress',...
    'StopTrainingCriteria','EpisodeReward',...
    'StopTrainingValue',1000);

if( doTraining )
    % Train the agent.
    trainingStats = train(agent,env,trainingOpts)
            ;
end

simOptions = rlSimulationOptions('MaxSteps',floor
        (1.2*trainingStats.EpisodeSteps(end)));
experience = sim(env,agent,simOptions);

t   = env.Ts*experience.Observation.
        SpacecraftStates.Time';
x   = experience.Observation.SpacecraftStates.Data
        ;
x   = reshape(x,2,size(x,3));
u   = experience.Action.torque.Data;
u   = [0 reshape(u,1,size(u,3))];
yL = {'\theta' '\omega' 'Angular Acceleration'};
s   = sprintf('Standard Deviation Min = %12.4f'
        ,...
  agent.AgentOptions.NoiseOptions.
          StandardDeviationMin);
TimeHistory(t,[x;u],yL,s);
```



Example M.5: One-axis reinforcement learning.

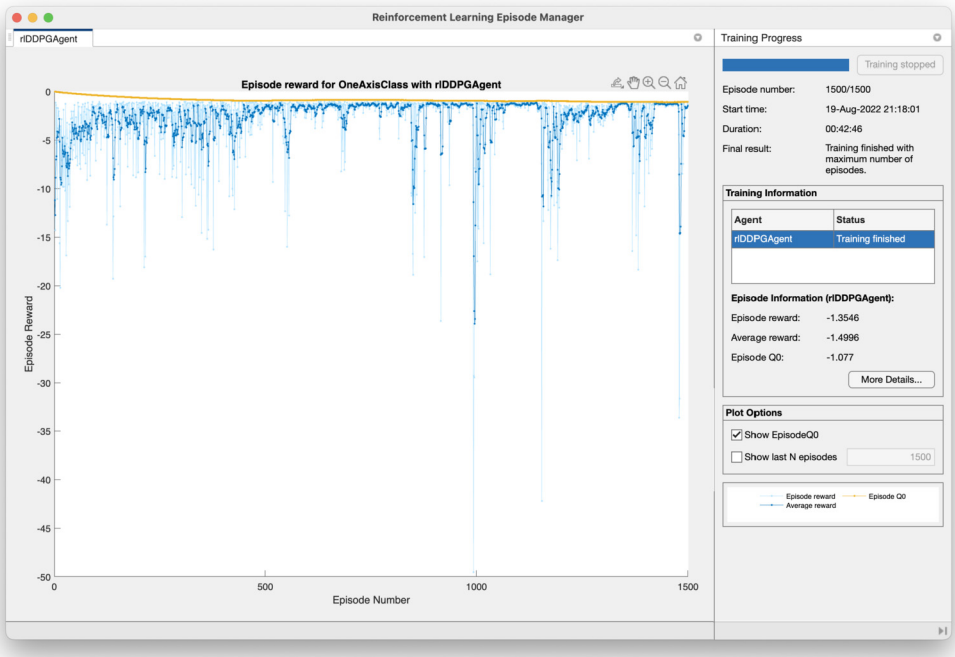


Figure M.15 Reinforcement-learning training window. The agent converges after 1500 episodes. The light blue line is the episode reward, the dark blue is the average reward since the start of training.

References

[1] G.A. Dorais, D. Kortenkamp, Deep Space One Remote Agent, https://personal.traclabs.com/~korten/tutorial/arc.pdf.

[2] D. Bernard, G. Dorais, C. Fry, E. Gamble, B. Kanefsky, J. Kurien, W. Millar, N. Muscettola, P. Nayak, B. Pell, K. Rajan, N. Rouquette, B. Smith, B. Williams, Design of the Remote Agent experiment for spacecraft autonomy, in: 1998 IEEE Aerospace Conference Proceedings (Cat. No. 98TH8339), vol. 2, 1998, pp. 259–281.

[3] P. Graham, ANSI Common Lisp, Prentice-Hall, 1995.

[4] M.D. Rayman, P. Varghese, D.H. Lehman, L.L. Livesay, Results from the Deep Space 1 technology validation mission, Acta Astronautica 47 (2) (2000) 475–487, Space an Integral Part of the Information Age.

[5] D.E. Goldberg, Genetic Algorithms in Search, Optimization, and Machine Learning, Addison-Wesley, 1988.

[6] I. Birbil, S.-C. Fang, An electromagnetism-like mechanism for global optimization, Journal of Global Optimization 25 (03) (2003) 263–282.

[7] L.R. Rere, M.I. Fanany, A.M. rymurthy, Simulated annealing algorithm for deep learning, Procedia Computer Science 72 (2015) 137–144.

[8] L. Bottou, F.E. Curtis, J. Nocedal, Optimization methods for large-scale machine learning, SIAM Review 60 (2016) 223–311.

[9] D. Bernard, R. Doyle, J. Riedel, N. Rouquette, J. Wyatt, M. Lowry, P. Nayak, Autonomy and software technology on NASA's Deep Space One, IEEE Intelligent Systems & Their Applications 14 (06) (1999) 10–15.
[10] D.S. Kolosa, A Reinforcement Learning Approach to Spacecraft Trajectory Optimization, Tech. Rep. Dissertations 3542, Western Michigan University, 2019.